

In The Name Of GOD

Internet of Things Laboratory Report

Bluetooth Communication & Ultrasonic Sensing

Project Title

*Ultrasonic Liquid Level Monitoring with HC-05 Bluetooth
and Arduino UNO*

Course: Internet of Things Lab

Instructor: Dr. Eisa ZarePoor

University/Department: IUST , Department of Computer Engineering

Date: Fall 2025

Team Information

#	Name & Surname	Student ID
1	Nazanin Mahmoodi	401522097
2	Mohammad Mahdi Sharaf- bayani	401521372

Contents

1	Introduction	2
2	Overall Device Architecture	2
3	Hardware and Software Tools	3
3.1	Hardware Components	3
3.2	Software Tools	3
3.3	Technologies and Protocols Used	3
4	Wiring and Workflow	4
4.1	Wiring Connections	4
4.1.1	HC-SR04 to Arduino UNO	4
4.1.2	HC-05 to Arduino UNO (Communication Mode)	4
4.1.3	Voltage Divider (Arduino D3 to HC-05 RXD)	4
4.2	Implementation Details (How the Device is Built and Configured)	4
4.2.1	Physical Installation	4
4.2.2	Software Setup and Pairing	5
4.2.3	Calibration and Operation Procedure	5
4.2.4	Computation Pipeline	5
5	Device Operation (How the System Works)	5
5.1	Distance Measurement Principle	5
5.2	Liquid Height and Level Calculation	6
5.3	Bluetooth Commands	6
5.4	Device Outputs	7
6	Appendix : Source Code	11

1 Introduction

This project implements a simple Internet of Things (IoT) liquid-level monitoring device using an Arduino UNO, an HC-SR04 ultrasonic distance sensor, and an HC-05 Bluetooth module. The system measures the distance between the sensor (mounted above a container) and the liquid surface, converts this distance into liquid height and percentage level using a calibration procedure, and transmits real-time readings to a smartphone over a serial Bluetooth link. In addition, the device supports a configurable low-level threshold that can be updated wirelessly via Bluetooth commands (e.g., T30 to set a 30% alert threshold). When the liquid level drops below the selected threshold, the system sends a warning message.

2 Overall Device Architecture

The proposed device follows a typical IoT edge-node architecture:

- 1) **Sensing Layer (Physical Layer):** The HC-SR04 sensor emits ultrasonic pulses and measures the echo time to estimate the distance to the liquid surface.
- 2) **Edge Processing Layer (Microcontroller Layer):** The Arduino UNO performs signal timing (pulse measurement), distance-to-level conversion, calibration management, threshold comparison, and message formatting.
- 3) **Communication Layer (Wireless Link):** The HC-05 module provides a Bluetooth serial channel (UART) between the Arduino and a smartphone.
- 4) **Application Layer (User Interface):** A smartphone terminal application displays live telemetry and allows the user to send control commands for calibration and threshold configuration.

Data Flow: HC-SR04 $\rightarrow$ Arduino UNO (compute) $\rightarrow$ HC-05 $\rightarrow$ Smartphone (display).

Control Flow: Smartphone (commands) $\rightarrow$ HC-05 $\rightarrow$ Arduino UNO (update parameters).

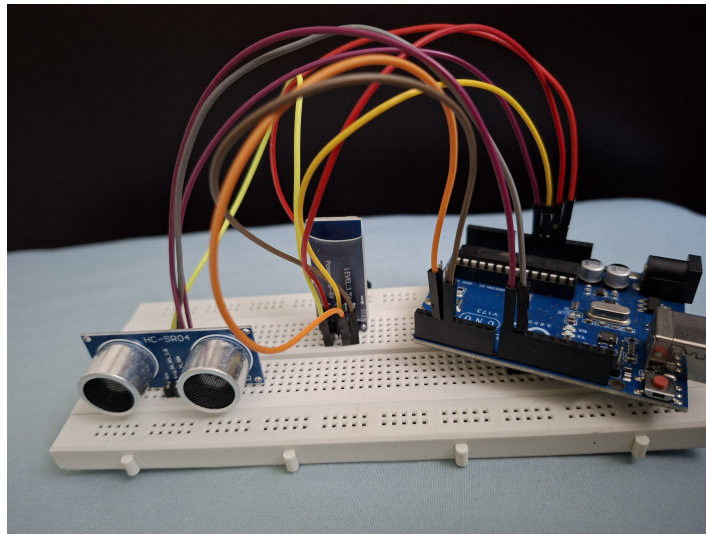


Figure 1: Complete wiring setup of the Arduino UNO, HC-05 Bluetooth module, and HC-SR04 ultrasonic sensor.

3 Hardware and Software Tools

3.1 Hardware Components

- **Arduino UNO:** Main microcontroller platform used for sensor reading, computation, and Bluetooth communication.
- **HC-SR04 Ultrasonic Sensor:** Measures the distance to the liquid surface using ultrasonic time-of-flight (trigger/echo).
- **HC-05 Bluetooth Module:** Provides Classic Bluetooth serial communication (UART) to a smartphone.
- **Voltage Divider for HC-05 RX:** Two resistors (e.g., $1\text{ k}\Omega$ and $2\text{ k}\Omega$) to reduce Arduino TX (5 V) to a safe level for HC-05 RX (3.3 V logic).
- **Jumper Wires and Breadboard:** For prototyping and wiring.
- **Smartphone with Bluetooth Terminal App:** Used to send commands and receive readings.

3.2 Software Tools

- **Arduino IDE:** Used to write, compile, and upload code to Arduino UNO.
- **Bluetooth Serial Terminal App:** Used to connect to HC-05, send calibration/threshold commands, and view the streamed data.

3.3 Technologies and Protocols Used

- **UART Serial Communication:** The Arduino UNO communicates with the HC-05 module using asynchronous UART via `SoftwareSerial` (RX/TX pins).
- **Bluetooth Classic SPP (Serial Port Profile):** The HC-05 exposes a serial link over Bluetooth, enabling a terminal app to send/receive text-based commands and telemetry.
- **Ultrasonic Time-of-Flight Ranging:** Distance is derived from the measured echo pulse duration based on the speed of sound.
- **Text-based Application Protocol (Custom Commands):** Simple ASCII commands (E, F, Txx, S, ?) implement calibration and configuration.

4 Wiring and Workflow

4.1 Wiring Connections

4.1.1 HC-SR04 to Arduino UNO

- **VCC** → Arduino **5V**
- **GND** → Arduino **GND**
- **TRIG** → Arduino **D9**
- **ECHO** → Arduino **D8**

4.1.2 HC-05 to Arduino UNO (Communication Mode)

- **VCC** → Arduino **5V**
- **GND** → Arduino **GND**
- **EN/KEY** → **Not connected** (kept disconnected for normal communication mode)
- **TXD (HC-05)** → Arduino **D2** (SoftwareSerial RX)
- **RXD (HC-05)** ← Arduino **D3** (SoftwareSerial TX) **through a voltage divider**

4.1.3 Voltage Divider (Arduino D3 to HC-05 RXD)

Because Arduino UNO outputs 5 V logic and HC-05 RX expects 3.3 V logic, a voltage divider is used:

- Arduino **D3** → **1 k Ω** → node
- node → **2 k Ω** → **GND**
- node → **HC-05 RXD**

4.2 Implementation Details (How the Device is Built and Configured)

This section describes the full implementation procedure from a practical perspective.

4.2.1 Physical Installation

- **Sensor placement:** The HC-SR04 should be mounted above the liquid container, facing downward toward the liquid surface. The sensor should not be tilted to minimize measurement errors due to angled reflections.
- **Minimum safe distance:** The sensor requires a minimum distance to function reliably (typically a few centimeters). Therefore, the “full” condition should still maintain a small air gap between the sensor and the liquid surface.
- **Cable management:** Ensure common ground between all modules (Arduino, HC-05, HC-SR04). Loose grounds are a frequent source of unstable readings.

4.2.2 Software Setup and Pairing

- Upload the firmware (Section 6) to the Arduino UNO using Arduino IDE.
- Pair the smartphone with the HC-05 module (Bluetooth settings).
- Open a Bluetooth serial terminal app and connect to the HC-05 device.

4.2.3 Calibration and Operation Procedure

The device uses two calibration points to map distance to level:

- 1) **Empty calibration (E):** When the tank is empty, send **E** to store **EMPTY_CM**. This represents the maximum measured distance to the surface.
- 2) **Full calibration (F):** When the tank is full (or at the maximum allowed level), send **F** to store **FULL_CM**. This represents the minimum measured distance to the surface.
- 3) **Threshold (Txx):** Optionally send **Txx** (e.g., **T30**) to set the low-level warning threshold as a percentage.
- 4) **Live monitoring:** The device continuously measures distance, computes height and percentage, and transmits formatted lines periodically (every 500 ms).

4.2.4 Computation Pipeline

- The sensor returns a distance d (cm) to the current liquid surface.
- The usable range is $R = \text{EMPTY_CM} - \text{FULL_CM}$.
- Liquid height is $h = \text{EMPTY_CM} - d$ and is clamped to $[0, R]$.
- Level percent is $\% = (h/R) \times 100$ and is clamped to $[0, 100]$.
- If $\% < \text{LEVEL_THRESHOLD}$, a warning is appended to the message.

5 Device Operation (How the System Works)

5.1 Distance Measurement Principle

The HC-SR04 measures distance using ultrasonic time-of-flight:

- 1) The Arduino drives the **TRIG** pin high for $10\ \mu\text{s}$ to emit an ultrasonic burst.
- 2) The sensor raises **ECHO** high for a duration proportional to the round-trip travel time of the sound wave.
- 3) The Arduino uses `pulseIn()` to measure the **ECHO** pulse width (microseconds), then converts it to distance in centimeters:

$$d(\text{cm}) = \frac{t(\mu\text{s}) \times 0.0343}{2}$$

where $0.0343\ \text{cm}/\mu\text{s}$ is the speed of sound (approximately $343\ \text{m/s}$) expressed in $\text{cm}/\mu\text{s}$.

5.2 Liquid Height and Level Calculation

Let:

- `EMPTY_CM` = distance from sensor to liquid surface when the tank is empty (largest distance),
- `FULL_CM` = distance from sensor to liquid surface when the tank is full (smallest distance),
- d = current measured distance.

Then:

- **Usable range:**

$$R = \text{EMPTY_CM} - \text{FULL_CM}$$

- **Current liquid height (from bottom reference):**

$$h = \text{EMPTY_CM} - d$$

- **Liquid level percentage:**

$$\% = \frac{h}{R} \times 100$$

The implementation clamps h and the percentage to safe bounds to avoid negative values or values above 100%.

5.3 Bluetooth Commands

- **E:** Capture the current distance as `EMPTY_CM`.
- **F:** Capture the current distance as `FULL_CM`.
- **Txx:** Set low-level threshold to **xx** percent (0–100).
- **S:** Print the current calibration values and threshold.
- **?:** Print command help.

5.4 Device Outputs

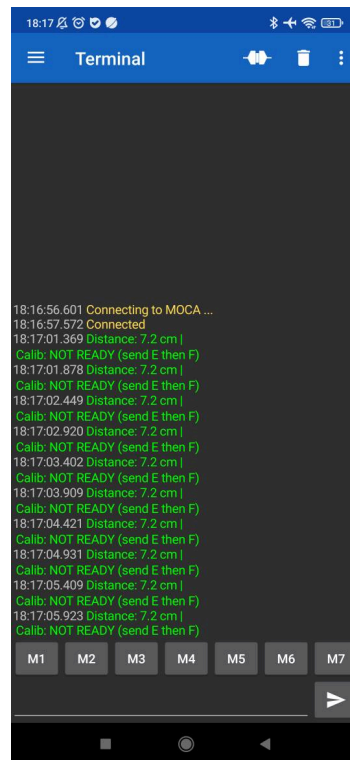


Figure 2: Live Bluetooth telemetry displaying ultrasonic sensor readings and indicating that calibration has not yet been performed.



Figure 3: Bluetooth serial output showing successful empty-tank calibration by storing the measured ultrasonic distance as the EMPTY reference value.



Figure 4: Bluetooth terminal output showing the full-tank calibration process and subsequent continuous liquid level measurements with threshold information.



Figure 5: Bluetooth terminal output showing successful modification of the low-level threshold to 60% followed by continuous liquid level reporting.

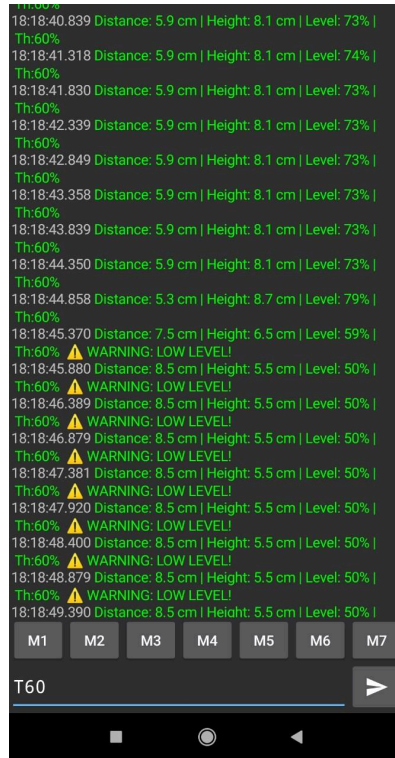


Figure 6: Bluetooth terminal output illustrating real-time measurements and repeated LOW LEVEL alerts when the liquid level falls beneath the configured threshold.



Figure 7: Bluetooth terminal output showing stored calibration parameters and current system configuration.

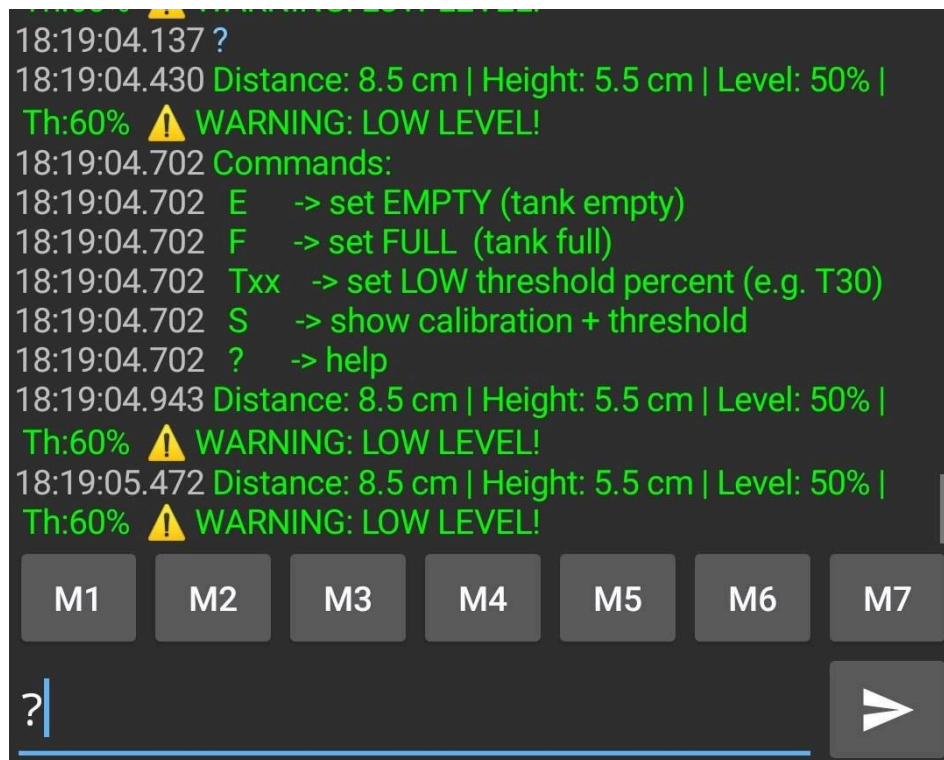


Figure 8: Bluetooth terminal output displaying the available Bluetooth command interface for controlling and monitoring the device.

6 Appendix : Source Code

Listing 1: Arduino UNO code with Bluetooth calibration (E/F), threshold configuration (Txx), and low-level warning.

```
1  #include <SoftwareSerial.h>           // Create a software
    UART for the HC-05
2
3  #define TRIG_PIN 9                   // HC-SR04 TRIG pin
    connected to D9
4  #define ECHO_PIN 8                   // HC-SR04 ECHO pin
    connected to D8
5
6  SoftwareSerial BT(2, 3);              // HC-05 UART: D2=RX (
    from HC-05 TX), D3=TX (to HC-05 RX)
7
8  // Calibration values (set by Bluetooth commands)
9  float EMPTY_CM = -1;                 // Distance when tank
    is empty (not set if -1)
10 float FULL_CM = -1;                  // Distance when tank
    is full (not set if -1)
11
12 // Low-level threshold (%) - changeable via Bluetooth command
    Txx
13 int LEVEL_THRESHOLD = 20;            // Default alert
    threshold (percent)
14
15 // Buffer for multi-character commands like T30
16 String cmd = "";                     // Accumulates incoming
    characters until end-of-line
17
18 // Clamp helper: bounds x into [a,b]
19 float clampf(float x, float a, float b) {
20     if (x < a) return a;              // Prevent values below
        minimum
21     if (x > b) return b;              // Prevent values above
        maximum
22     return x;                         // Otherwise return x
23 }
24
25 // Reads distance in centimeters using HC-SR04 time-of-flight
26 float readDistanceCm() {
27     digitalWrite(TRIG_PIN, LOW);      // Ensure TRIG is low
        before pulse
28     delayMicroseconds(2);              // Short settling delay
29     digitalWrite(TRIG_PIN, HIGH);      // Start trigger pulse
30     delayMicroseconds(10);             // Hold for 10us (HC-
        SR04 requirement)
31     digitalWrite(TRIG_PIN, LOW);      // End trigger pulse
32
```

```

33     long duration = pulseIn(ECHO_PIN, HIGH, 30000); // Measure
        echo pulse width (timeout 30ms)
34     if (duration == 0) return -1;                // No echo received (
        timeout)
35
36     return duration * 0.0343f / 2.0f;           // Convert us -> cm (
        speed of sound) and divide by 2
37 }
38
39 // Prints supported commands to the Bluetooth terminal
40 void printHelp() {
41     BT.println("Commands:");
42     BT.println("  E      -> set EMPTY (tank empty)");
43     BT.println("  F      -> set FULL  (tank full)");
44     BT.println("  Txx    -> set LOW threshold percent (e.g. T30)");
45     ;
46     BT.println("  S      -> show calibration + threshold");
47     BT.println("  ?      -> help");
48 }
49
50 // Shows calibration values and current threshold
51 void showCalibration() {
52     BT.print("EMPTY_CM = ");
53     if (EMPTY_CM < 0) BT.println("NOT SET");           // EMPTY not
        calibrated yet
54     else { BT.print(EMPTY_CM, 1); BT.println(" cm"); }
55
56     BT.print("FULL_CM = ");
57     if (FULL_CM < 0) BT.println("NOT SET");           // FULL not
        calibrated yet
58     else { BT.print(FULL_CM, 1); BT.println(" cm"); }
59
60     if (EMPTY_CM >= 0 && FULL_CM >= 0) {
61         BT.print("Range (EMPTY-FULL) = ");
62         BT.print(EMPTY_CM - FULL_CM, 1);
63         BT.println(" cm");
64     }
65
66     BT.print("LOW LEVEL THRESHOLD = ");
67     BT.print(LEVEL_THRESHOLD);
68     BT.println("%");
69 }
70
71 // Parses and executes a full command string
72 void handleCommand(const String &c) {
73     if (c.length() == 0) return;                // Ignore empty
        commands
74
75     // Single-letter commands
76     if (c == "E" || c == "e") {                // Store empty-tank
        distance

```

```

76     float d = readDistanceCm();
77     if (d < 0) BT.println("ERR: No echo. Try again.");
78     else {
79         EMPTY_CM = d;
80         BT.print("OK: EMPTY set to ");
81         BT.print(EMPTY_CM, 1);
82         BT.println(" cm");
83     }
84     return;
85 }
86
87 if (c == "F" || c == "f") {                // Store full-tank
88     distance
89     float d = readDistanceCm();
90     if (d < 0) BT.println("ERR: No echo. Try again.");
91     else {
92         FULL_CM = d;
93         BT.print("OK: FULL set to ");
94         BT.print(FULL_CM, 1);
95         BT.println(" cm");
96     }
97     return;
98 }
99
100 if (c == "S" || c == "s") {                // Print current
101     configuration
102     showCalibration();
103     return;
104 }
105
106 if (c == "?") {                            // Print help
107     printHelp();
108     return;
109 }
110
111 // Threshold command: Txx (e.g., T30)
112 if (c.length() >= 2 && (c[0] == 'T' || c[0] == 't')) {
113     String numPart = c.substring(1);      // Extract digits after
114     'T'
115     numPart.trim();                       // Remove any spaces
116     int val = numPart.toInt();            // Convert to integer
117
118     if (val < 0 || val > 100) {            // Validate range
119         BT.println("ERR: Threshold must be 0..100 (e.g. T30)");
120         return;
121     }
122
123     LEVEL_THRESHOLD = val;                // Update threshold
124     BT.print("OK: LOW threshold set to ");
125     BT.print(LEVEL_THRESHOLD);
126     BT.println("%");

```

```

124     return;
125 }
126
127 BT.print("Unknown cmd: ");           // Unknown command
    handler
128 BT.println(c);
129 BT.println("Send ? for help");
130 }
131
132 void setup() {
133     pinMode(TRIG_PIN, OUTPUT);        // Configure TRIG as
        output
134     pinMode(ECHO_PIN, INPUT);         // Configure ECHO as
        input
135
136     BT.begin(9600);                   // Start Bluetooth
        serial (typical HC-05 SPP baud)
137     BT.println("Ultrasonic Level Monitor");
138     printHelp();                      // Print command usage
        on startup
139     showCalibration();                // Show initial
        calibration state
140 }
141
142 void loop() {
143     // ---- Read command from phone until end-of-line ----
144     while (BT.available()) {          // Read all available
        characters
145         char ch = BT.read();
146
147         if (ch == '\n' || ch == '\r') { // End-of-line =>
            command complete
148             if (cmd.length() > 0) {
149                 handleCommand(cmd);    // Execute the command
150                 cmd = "";              // Clear buffer after
                    processing
151             }
152         } else {
153             if (cmd.length() < 20) cmd += ch; // Accumulate command;
                cap length for safety
154         }
155     }
156
157     // ---- Send status every 500 ms ----
158     static unsigned long lastSend = 0; // Timestamp of last
        telemetry send
159     if (millis() - lastSend >= 500) {
160         lastSend = millis();
161
162         float d = readDistanceCm();    // Current distance to
            liquid surface

```

```

163     if (d < 0) {
164         BT.println("Distance: NO ECHO");    // Sensor reading
165         failed
166         return;
167     }
168
169     // Not calibrated => show distance only
170     if (EMPTY_CM < 0 || FULL_CM < 0 || (EMPTY_CM - FULL_CM) <
171         1.0f) {
172         BT.print("Distance: ");
173         BT.print(d, 1);
174         BT.println(" cm | Calib: NOT READY (send E then F)");
175         return;
176     }
177
178     float range = (EMPTY_CM - FULL_CM);    // Usable distance
179     range between empty and full
180
181     float height = EMPTY_CM - d;           // Liquid height
182     relative to empty reference
183     height = clampf(height, 0, range);    // Clamp to [0, range]
184
185     float pct = (height / range) * 100.0f; // Convert height to
186     percentage
187     pct = clampf(pct, 0, 100);           // Clamp to [0, 100]
188
189     // ---- Transmit formatted telemetry ----
190     BT.print("Distance: ");
191     BT.print(d, 1);
192     BT.print(" cm | Height: ");
193     BT.print(height, 1);
194     BT.print(" cm | Level: ");
195     BT.print(pct, 0);
196     BT.print("% | Th:");
197     BT.print(LEVEL_THRESHOLD);
198     BT.print("%");
199
200     // Low-level warning
201     if ((int)pct < LEVEL_THRESHOLD) {
202         BT.print(" WARNING: LOW LEVEL!");
203     }
204
205     BT.println();    // End of telemetry
206     line
207 }
208 }

```